

Báo cáo 3: Giải thích Code Sinh ra Metrics và Kết quả

Walkthrough chi tiết `src/run/main.py` — EducationQ Framework

Seminar Deep Learning

2026

Mục lục

1	1. Kiến trúc tổng thể	2
2	2. Cấu hình — <code>EvalConfig</code> (dòng 38–79)	2
3	3. TeacherLLM — Sinh câu hỏi Socratic	3
3.1	3.1 Khởi tạo (dòng 196–266)	3
3.2	3.2 Sinh câu hỏi — <code>generate_question()</code> (dòng 268–336)	3
3.3	3.3 Retry logic (dòng 315–336)	4
4	4. StudentLLM — Làm test và trả lời	5
4.1	4.1 Trả lời trong interaction — <code>answer_question()</code> (dòng 386–452)	5
4.2	4.2 Làm bài test — <code>take_test()</code> (dòng 454–617)	5
4.3	4.3 Trích xuất đáp án — <code>extract_answer()</code> (dòng 604–617)	5
5	5. EvaluatorLLM — Tính metrics	6
5.1	5.1 Tính accuracy — <code>calculate_accuracy()</code> (dòng 713–734)	6
5.2	5.2 Đánh giá định tính — <code>over_interaction_analysis()</code> (dòng 910–996)	6
6	6. EvalManager — Orchestrator	7
6.1	6.1 <code>_run_pretest()</code> (dòng 1477–1569)	7
6.2	6.2 <code>_run_interactions()</code> (dòng 1571–1650)	8
6.3	6.3 <code>_process_question()</code> — vòng lặp dạy học (dòng 1652–1682)	8
6.4	6.4 <code>_perform_evaluation()</code> — tính ALG (dòng 1754–1809)	9
7	7. Luồng dữ liệu end-to-end	10
8	8. Kết nối giữa code và metrics	11
9	Tài liệu tham khảo	11

1 1. Kiến trúc tổng thể

File `src/run/main.py` (~2,200 dòng) chứa toàn bộ framework. Cấu trúc class:

```

EvalConfig          ← đọc YAML config
BaseLLM             ← base class cho tất cả agents
├── TeacherLLM      ← sinh câu hỏi Socratic
├── StudentLLM       ← làm test + trả lời teacher
└── EvaluatorLLM     ← tính accuracy + phân tích định tính
BaseDataset         ← base class cho datasets
├── MMLU_PRO         ← loader cho MMLU-Pro
├── GPQA             ← loader cho GPQA
└── AGIEVAL          ← loader cho AGIEval
EvalManager         ← orchestrator toàn bộ pipeline

```

Pipeline chính được khởi động qua `main()` và `EvalManager.run_complete_eval()`:

```

# main.py: run_complete_eval() - dòng 1470-1475
def run_complete_eval(self):
    pretest_results      = self._run_pretest()
    interaction_results   = self._run_interactions(pretest_results)
    posttest_results     = self._run_posttest(interaction_results)
    evaluation_results    = self._perform_evaluation(posttest_results)
    return evaluation_results

```

2 2. Cấu hình — EvalConfig (dòng 38–79)

```

class EvalConfig:
    def __init__(self, config_data: Dict[str, Any]):
        self.num_interactions = config_data.get("NUM_INTERACTIONS", 5)
        self.teacher_configs  = config_data.get("TEACHER_CONFIGS", [])
        self.student_configs  = config_data.get("STUDENT_CONFIGS", [])
        self.evaluator_config = config_data.get("EVALUATOR_CONFIG", {})
        ...

    @classmethod
    def from_yaml(cls, config_path: str):
        with open(config_path, "r") as f:
            config_data = yaml.safe_load(f)
        return cls(config_data)

```

Điểm quan trọng từ config_teacher0shot_gpqa_diamond.yaml:

- NUM_INTERACTIONS: 5 → mỗi câu hỏi có **5 vòng** hội thoại teacher–student
 - use_few_shot: false → teacher dạy **zero-shot** (không có ví dụ mẫu)
 - temperature: 0.0 → output **deterministic** (cùng input → cùng output)
 - include_pretest_info: true → student **thấy câu trả lời pretest của mình** khi làm posttest
-

3 TeacherLLM — Sinh câu hỏi Socratic

3.1 Khởi tạo (dòng 196–266)

```
class TeacherLLM(BaseLLM):
    def __init__(self, ...,
        recommended_question_token_limit: int = 150,
        recommended_education_theory: Optional[str] = None,
        ...):
        self.recommended_question_token_limit = recommended_question_token_limit
        # recommended_education_theory = None trong config thực nghiệm
        # → không ràng buộc lý thuyết sư phạm cụ thể
```

Tham số recommended_education_theory = null trong config GPQA có nghĩa teacher **tự do chọn** phương pháp sư phạm, không bị ép dùng ZPD hay Bloom cụ thể.

3.2 Sinh câu hỏi — generate_question() (dòng 268–336)

```
def generate_question(self, category, pre_test_results,
    interaction_history, current_round, total_rounds,
    few_shot_examples=None):
```

```
    system_message = f"""You are an expert teacher in {category} \
dedicated to enhancing the student's understanding after analyzing \
the student's response to a pre-test.
Your task is to ask {total_rounds} rounds of relevant, \
thought-provoking questions to the student.
You should ask one new question per round (and if needed, provide \
necessary corrections or feedback for the student's previous answers),
each under {self.recommended_question_token_limit} tokens, \
without revealing the correct answers ..."""
```

System prompt phân tích:

- Teacher được cho biết *kết quả pretest* của student (câu hỏi, lý luận, đáp án đúng/sai)
- Teacher **không được tiết lộ đáp án** — buộc phải dùng Socratic questioning
- Token limit 150 → câu hỏi ngắn gọn, tập trung
- Lịch sử tương tác được truyền vào để teacher **không hỏi lại** điều đã hỏi

```
# Đưa pre-test info vào system message
pre_test_info = "\n\nHere are the pre-test results:\n"
for r in pre_test_results:
    pre_test_info += f"""
        Question: {r['question']}
        Student's Answer: {r['model_prediction']}
        Correct: {"Correct" if r['correct_answer']==r['model_prediction']
                  else "Incorrect"}
    """

# Đưa lịch sử interaction vào messages
for interaction in interaction_history:
    messages.append({"role": "assistant",
                    "content": f"Teacher: {interaction['question']}"})
    messages.append({"role": "user",
                    "content": f"Student: {interaction['answer']}"})
```

3.3 3.3 Retry logic (dòng 315–336)

```
retry_count = 0
while retry_count < self.question_retries: # = 5 theo config
    response = self.generate_response(messages)
    response_tokens = self.count_tokens(response)
    max_allowed = max(self.max_tokens * 0.8,
                      self.recommended_question_token_limit)

    if 0 < response_tokens <= max_allowed:
        return response # OK
    # Nếu quá dài → retry
    retry_count += 1
```

Khi response vượt 80% max_tokens và dài hơn 150 tokens → retry để tránh câu hỏi quá dài làm loãng focus.

4 4. StudentLLM — Làm test và trả lời

4.1 4.1 Trả lời trong interaction — `answer_question()` (dòng 386–452)

```
def answer_question(self, category, question, interaction_history,
                    pre_test_results=None, few_shot_examples=None):
    system_message = f"You are a student focusing on {category}. \
Analyze the question carefully, explain your thought process \
({self.recommended_answer_token_limit} tokens or less) ..."
```

Student nhận toàn bộ lịch sử interaction → **nhớ được context** các vòng hỏi trước.

4.2 4.2 Làm bài test — `take_test()` (dòng 454–617)

```
def take_test(self, test_data, few_shot_cot_examples=[],
              interaction_history=None, pre_test_results=None):

    if interaction_history:
        test_question = "Based on your current understanding after \
the conversation with the teacher, rethink step by step and then \
finish your answer with 'the answer is (X)':\n"
    else:
        test_question = "" # pretest: không có context

    test_question += self.format_question(question["question"],
                                         question["options"])
```

Sự khác biệt pretest vs posttest:

	Pretest	Posttest
interaction_history	None	Toàn bộ 5 vòng hội thoại
pre_test_results	None	Câu trả lời pretest của chính student
Prompt	Cold start	“Rethink based on your conversation”

4.3 4.3 Trích xuất đáp án — `extract_answer()` (dòng 604–617)

```
@staticmethod
def extract_answer(text: str) -> str:
    pattern = r"answer is \((?([A-J])\))?"
    match = re.search(pattern, text)
    if match:
        return match.group(1)
```

```

else:
    # Fallback: tìm chữ cái cuối cùng xuất hiện
    pattern = r"\b[A-J]\b(?!.*\b[A-J]\b)"
    match = re.search(pattern, text, re.DOTALL)
    return match.group(0) if match else "R" # "R" = invalid

```

Answer extraction dùng regex 3 tầng để bắt các format trả lời khác nhau của LLM.

5. EvaluatorLLM — Tính metrics

5.1 5.1 Tính accuracy — `calculate_accuracy()` (dòng 713–734)

```

def calculate_accuracy(self, test_responses):
    category_scores = defaultdict(lambda: {"correct": 0, "total": 0})
    total_correct = 0

    for response in test_responses:
        category = response["category"]
        category_scores[category]["total"] += 1
        if response["correct_answer"] == response["model_prediction"]:
            category_scores[category]["correct"] += 1
            total_correct += 1

    return {
        "category_accuracy": {cat: scores["correct"]/scores["total"]
                               for cat, scores in category_scores.items()},
        "overall_accuracy": total_correct / len(test_responses),
    }

```

Hàm này tạo ra scores được lưu trong mỗi episode tại `pre_test.scores` và `post_test.scores`.

5.2 5.2 Đánh giá định tính — `over_interaction_analysis()` (dòng 910–996)

```

def over_interaction_analysis(self, question_id, category,
                             pre_test_result,
                             teacher_1_name, teacher_1_interaction,
                             teacher_2_name, teacher_2_interaction):
    # Blind anonymization
    if random.choice([True, False]):
        prompt_teacher_a = (teacher_1_name, teacher_1_interaction)

```

```

        prompt_teacher_b = (teacher_2_name, teacher_2_interaction)
    else:
        prompt_teacher_a = (teacher_2_name, teacher_2_interaction)
        prompt_teacher_b = (teacher_1_name, teacher_1_interaction)

    instruction = """You are an expert in educational assessment...
1. Assessment Effectiveness: ...
2. Questioning Effectiveness: ...
3. Feedback Effectiveness: ...
4. Instructional Adaptation Effectiveness: ...
5. Learning Objective Achievement Effectiveness: ..."""

```

Structured output bằng JSON Schema (dòng 1047–1064):

```

def generate_response(self, messages, schema):
    response = self.client.chat.completions.create(
        model=self.model,
        messages=messages,
        response_format={
            "type": "json_schema",
            "json_schema": {
                "name": "evaluation_response",
                "schema": schema,      # enforce structure
                "strict": True         # no extra fields
            }
        }
    )
    return response.choices[0].message.content

```

OpenAI `json_schema` mode đảm bảo output **luôn** có đúng format — không bao giờ trả về text tự do.

6. EvalManager — Orchestrator

6.1 `_run_pretest()` (dòng 1477–1569)

```

def _run_pretest(self):
    with ThreadPoolExecutor(max_workers=CONFIG["PARALLEL_TASKS"]) as executor:
        for category, questions in self.test_data.items():
            for question in questions:
                future = executor.submit(student.take_test,

```

```

[question],
few_shot_cot_examples)
# interaction_history=None → pretest mode

```

Parallel execution: tất cả câu hỏi chạy song song bằng `ThreadPoolExecutor.PARALLEL_TASKS = 5` trong default config.

6.2 6.2 _run_interactions() (dòng 1571–1650)

```

def _run_interactions(self, pretest_results):
    for teacher in self.teachers:
        for student_name, student_data in pretest_results.items():
            for category, questions in self.test_data.items():
                future = executor.submit(
                    self._process_question,
                    teacher, student, question,
                    few_shot_cot_examples,
                    self.config.num_interactions, # = 5
                    pretest_result['responses'],
                    pretest_result['scores']
                )

```

6.3 6.3 _process_question() — vòng lặp dạy học (dòng 1652–1682)

```

def _process_question(self, teacher, student, question, ...):
    interaction_history = []

    for i in range(num_interactions): # 5 vòng
        teacher_question = teacher.generate_question(
            category, pre_test_result,
            interaction_history,
            current_round=i+1,
            total_rounds=num_interactions
        )
        student_answer = student.answer_question(
            category, teacher_question,
            interaction_history, pre_test_result
        )
        interaction_history.append({
            "question": teacher_question,
            "answer": student_answer

```



```

    })

    return {
        "category": category,
        "question_id": question_id,
        "pre_test": {"responses": pre_test_result, "scores": pre_test_score},
        "interaction": interaction_history,    # 5 turns
    }

```

6.4 6.4_perform_evaluation() — tính ALG (dòng 1754–1809)

```

def _perform_evaluation(self, results):
    for teacher_name, teacher_data in results.items():
        for question_id, question_data in student_data.items():
            category = question_data['category']
            pre_test = question_data['pre_test']['responses'][0]
            post_test = question_data['post_test']['responses'][0]

            # Đếm số câu đúng theo category
            category_results[category]['total'] += 1
            if pre_test['model_prediction'] == pre_test['correct_answer']:
                category_results[category]['pre_test_correct'] += 1
            if post_test['model_prediction'] == post_test['correct_answer']:
                category_results[category]['post_test_correct'] += 1

            # Tính ALG cho từng category
            for category, results in category_results.items():
                pre_acc = results['pre_test_correct'] / results['total']
                post_acc = results['post_test_correct'] / results['total']
                progress = post_acc - pre_acc    # ← ALG (dòng 1782)

            # Tính overall ALG
            evaluation[teacher][student]['overall'] = {
                "pre_test_accuracy": total_pre / total_questions,
                "post_test_accuracy": total_post / total_questions,
                "progress": (total_post - total_pre) / total_questions,
            }

```

7 7. Luồng dữ liệu end-to-end

```

config.yaml
|
▼
EvalConfig.from_yaml()
|
▼
EvalManager.__init__()
  ├── _create_teachers()    → [TeacherLLM, ...]
  ├── _create_students()   → [StudentLLM, ...]
  ├── _create_evaluator()  → EvaluatorLLM
  └── _load_datasets()     → {category: [questions]}
|
▼
_run_pretest()
  ├── StudentLLM.take_test(interaction_history=None)
  ├── EvaluatorLLM.calculate_accuracy()
  └── Luu: pretest_results_{version}_{ts}.json
|
▼
_run_interactions()
  ├── _process_question() × (N_teachers × N_questions)
  │   ├── TeacherLLM.generate_question() × 5 rounds
  │   └── StudentLLM.answer_question()   × 5 rounds
  └── Luu: pretest_interaction_results_{version}_{ts}.json
|
▼
_run_posttest()
  ├── StudentLLM.take_test(interaction_history=[5 turns])
  ├── EvaluatorLLM.calculate_accuracy()
  └── Luu: pretest_interaction_posttest_results_{version}_{ts}.json
|
▼
_perform_evaluation()
  ├── progress = post_acc - pre_acc (ALG)
  └── Luu: evaluation_results_{version}_{ts}.json

```

8. Kết nối giữa code và metrics

Metric	Hàm tính	Dòng code
Pre-test accuracy	<code>calculate_accuracy()</code>	713–734
Post-test accuracy	<code>calculate_accuracy()</code>	713–734
ALG (progress)	<code>_perform_evaluation()</code>	782
Qualitative score	<code>over_interaction_analysis()</code>	910–996
Teacher Q score	<code>teacher_questions_analysis()</code>	736–821
Student R score	<code>student_responses_analysis()</code>	823–908
De-anonymization	<code>deanonymize_evaluation()</code>	1025–1045

9 Tài liệu tham khảo

Shi, Y., Liang, R., & Xu, Y. (2025). EducationQ: Multi-Agent Framework for Evaluating LLM Teaching Capability. *Proceedings of ACL 2025*, pp. 32799–32828.

Source code: `src/run/main.py` — <https://github.com/iam-Dylan/education-q>